Module 6

Exercise 3 (Postgress & NodeJs)

Q1.) Create Functions for CRUD Operations:

A1.)

Module6 > Exercise3 > db.js > pool > user

```javascript
const {Pool}=require("pg");
require('dotenv').config();
const pool=new Pool({
    user:process.env.DB_USER,
    host:process.env.HOST,
    database:process.env.DATABASE,
    password:process.env.PASSWORD,
    port:process.env.PORT

});

module.exports=pool
```

a. Create: Implement a function to add a new student to the `students` table.

Module6 > Exercise3 > studentModel.js > <unknown> > deleteStudent

```javascript
const pool = require("./db");


async function addStudent(name, grade) {
  try {
    if (!name || !grade) {
      throw new Error("Invalid student data");
    }

    const result = await pool.query(
      "INSERT INTO students(name, grade) VALUES($1,$2) RETURNING *",
      [name, grade]
    );

    console.log("Student Added:", result.rows[0]);
  } catch (err) {
    console.error("Create Error:", err.message);
  }
}
```

b. Read: Implement a function to fetch all students from the table.

```javascript
async function getStudents() {
  try {
    const result = await pool.query("SELECT * FROM students");

    console.log("All Students:");
    console.table(result.rows);
  } catch (err) {
    console.error("Read Error:", err.message);
  }
}
```

c. Update: Implement a function to update a student's grade based on their `id`.

```
32  async function updateStudentGrade(id, grade) {
33    try {
34      const result = await pool.query(
35        "UPDATE students SET grade=$1 WHERE id=$2 RETURNING *",
36        [grade, id]
37      );
38
39      if (result.rowCount === 0) {
40        console.log("Student not found");
41      } else {
42        console.log("Updated:", result.rows[0]);
43      }
44    } catch (err) {
45      console.error("Update Error:", err.message);
46    }
47  }
```

d. Delete: Implement a function to delete a student by `id`.

```
49  async function deleteStudent(id) {
50    try {
51      const result = await pool.query(
52        "DELETE FROM students WHERE id=$1 RETURNING *",
53        [id]
54      );
55
56      if (result.rowCount === 0) {
57        console.log("Student not found");
58      } else {
59        console.log("Deleted:", result.rows[0]);
60      }
61    } catch (err) {
62      console.error("Delete Error:", err.message);
63    }
64  }
```

Q2.) Test the CRUD Operations:

Create a `main.js` file where you:

i. Add a student.
ii. Retrieve all students.
iii. Update a student's grade.
iv. Delete a student.

A2.)

```javascript
const student = require("./studentModel");

async function runCRUD() {

  await student.addStudent("Dhruv", "A");
  await student.addStudent("Harsh", "A");
  await student.addStudent("Aman", "B");
  await student.addStudent("Sourav", "B+");

  await student.getStudents();

  await student.updateStudentGrade(1, "A+");

  await student.deleteStudent(3);

  await student.getStudents();
}

runCRUD();
```

Output

```
Student Added: { id: 1, name: 'Dhruv', grade: 'A' }
Student Added: { id: 2, name: 'Harsh', grade: 'A' }
Student Added: { id: 3, name: 'Aman', grade: 'B' }
Student Added: { id: 4, name: 'Sourav', grade: 'B+' }
All Students:
```

(index)	id	name	grade
0	1	'Dhruv'	'A'
1	2	'Harsh'	'A'
2	3	'Aman'	'B'
3	4	'Sourav'	'B+'

```
Updated: { id: 1, name: 'Dhruv', grade: 'A+' }
Deleted: { id: 3, name: 'Aman', grade: 'B' }
All Students:
```

(index)	id	name	grade
0	2	'Harsh'	'A'
1	4	'Sourav'	'B+'
2	1	'Dhruv'	'A+'